

Educação
Profissional
Paulista

Técnico em
Ciência de
Dados

Manipulação de arquivos

Leitura de arquivos em Python

Aula 1

Código da aula: [DADOS]ANO1C2B2S16A1



Objetivos da Aula

- Entender os conceitos básicos de manipulação de arquivo, abrir, ler e escrever;
- Conhecer o tratamento de exceções em Python.



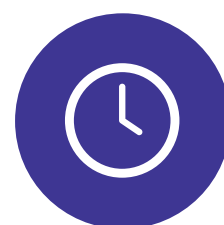
Competências da Unidade (Técnicas e Socioemocionais)

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências;
- Cooperar com profissionais da área, incluindo cientistas e engenheiros de dados, e atuar em equipes diversificadas, colaborando com parceiros, supervisores e clientes.



Recursos Didáticos

- Recurso audiovisual para a exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da Aula

50 minutos.

Exposição

Armazenamento dos dados

Desenvolvemos um incrível programa de **agenda de contatos** no qual era possível cadastrar, alterar, excluir e visualizar contatos.

Parabéns! Foi um excelente trabalho!

Mas e agora?

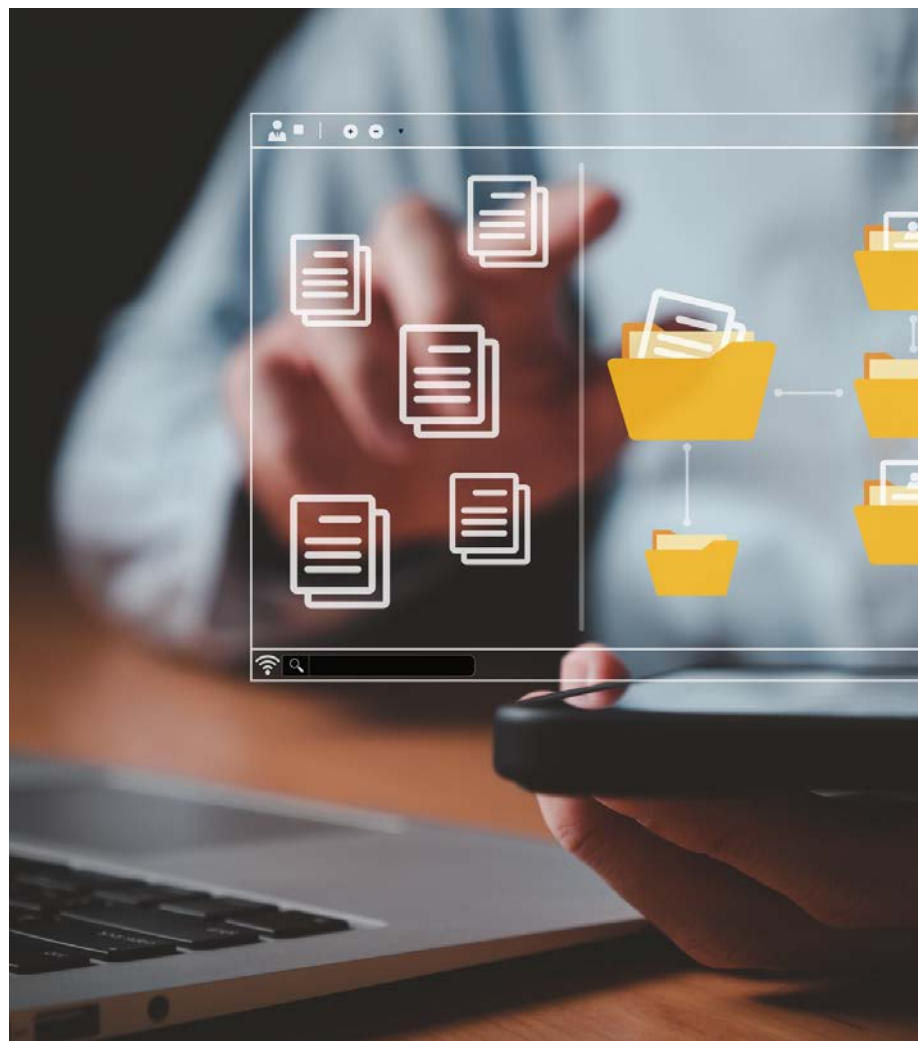
Após a finalização do programa, você já se perguntou para onde os dados vão? Será que se tornam **arquivos**?

Será que estamos nos esquecendo de algo?
Como podemos aprimorar ainda mais nossa aplicação?



© Getty Images

Exposição



© Getty Images

Visão geral: arquivos

Anteriormente, dados em programas eram perdidos quando eram fechados, necessitando de reexecução para recuperação.

A solução veio a partir de salvar dados em arquivos, em SSDs, HDDs localmente ou na nuvem, garantindo o uso eficiente dessas informações em diversas execuções.

O que é um arquivo?

Arquivo é uma **unidade de armazenamento** que contém dados ou informações de texto ou binários.



Tome nota

Os arquivos são utilizados para armazenar e organizar dados de maneira persistente, permitindo que as informações sejam salvas e acessadas posteriormente.

Tipos de arquivos

Existem diferentes tipos de arquivos, mas os dois principais são:



Arquivos de texto

- Armazenam dados em formato de **texto simples**;
- Podem ser lidos e interpretados por humanos.
- Exemplos incluem arquivos .txt, .csv e .json.



Arquivos binários

- Armazenam dados em um formato mais **compacto e eficiente**.
- Não são facilmente interpretados por humanos.
- Exemplos incluem arquivos de imagem (.png, .jpg etc.), áudio e executáveis.

Tipos de arquivos



TXT: é um formato de arquivo de texto simples que contém apenas caracteres sem formatação. Pode ser aberto por qualquer editor de texto, como o Bloco de Notas ou o WordPad. É usado para armazenar informações simples, como notas, listas ou códigos-fonte.



CSV: formato que separa dados com vírgulas, perfeito para planilhas e bancos de dados. Cada linha é um registro e é compatível com Excel e Google Sheets, facilitando o armazenamento e a exportação de dados como contatos e relatórios.

Imagens: © Getty Images

Exposição

Tipos de arquivos



© Getty Images

JSON: formato de arquivo baseado em JavaScript, com dados em pares chave-valor. Fácil de ler e escrever por humanos e máquinas, é compatível com softwares como Visual Studio Code e Notepad++. Utilizado para gerenciar dados em aplicativos, APIs, serviços web e configurações.



Elaborado especialmente para o curso com imagem © Getty Images.

PY: formato de arquivo de script Python, uma linguagem de programação orientada a objetos. Requer interpretador Python para execução e é usado em programação de servidores web e aplicações. Editável em IDEs como Notepad++, Sublime Text e PyCharm.

Tipos de arquivos



JPG: formato de imagem que comprime arquivos com perda, sem suportar transparência, mas possibilitando ajuste de compressão. Abre em softwares como Paint e Photoshop, sendo usado para fotos, ilustrações e pôsteres.



PNG: formato de imagem que oferece compressão sem perda, suporta transparência e diversas cores. Funciona em programas como Paint e Photoshop e é ideal para armazenar e mostrar imagens, incluindo logotipos e ícones.

Exposição

Funções embutidas

Já vimos algumas funções embutidas (built-in) em Python.

São funções que estão disponíveis no interpretador da linguagem, ou seja, são integradas (built-in) e não requerem a importação de módulos externos para serem utilizadas.

E a função **open()**?

A função **open()** é uma função built-in que é usada para abrir um arquivo. Ela viabiliza trabalhar com arquivos, seja para leitura, escrita ou outras operações.

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	str() sum() super()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	T tuple() type()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	V vars()
			Z zip()
			__import__()

Reprodução - PYTHON. Built-in Functions. Disponível em: <https://docs.python.org/3/library/functions.html>
Acesso em: 5 abr. 2024.

Abertura de arquivo

Para abrir um arquivo, é usada a função **open()**.

Possui dois argumentos: o caminho do arquivo e o modo de operação.

```
# Exemplo de abertura de arquivo para leitura  
with open('nome_do_arquivo.txt', 'r') as arquivo:  
    # código para manipular o arquivo  
pass # o bloco with fecha automaticamente o arquivo quando termina
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Modos de abertura de arquivo:

- 'r'**: leitura (padrão).
- 'w'**: escrita (cria um novo arquivo ou sobrescreve o existente).
- 'a'**: anexar (adiciona conteúdo ao final do arquivo).
- 'b'**: modo binário (por exemplo, 'rb' para leitura binária).
- 't'**: modo de texto (padrão).
- '+'**: atualiza (ao combinar com outros modos, possibilita a alteração do arquivo existente).

Leitura de arquivo

Para ler o conteúdo de um arquivo, não é suficiente apenas abrir o arquivo com a função **open()**. Utiliza-se a função **read()** para carregar o conteúdo do arquivo em uma string.

```
with open('arquivo.txt', 'r') as arquivo:
    conteudo = arquivo.read() # lê todo o conteúdo
    linhas = arquivo.readlines() # retorna uma lista de linhas
    for linha in arquivo:
        # processa cada linha individualmente
    pass
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Modos de leitura:

- **read()**: lê todo o conteúdo do arquivo como uma única string.
- **readlines()**: retorna uma lista de strings em que cada elemento é uma linha do arquivo.
- Iterar sobre o arquivo diretamente usando um loop **for**: lê o arquivo linha por linha.

Escrita em arquivo

A escrita em arquivo Python envolve a gravação de dados em um arquivo no próprio sistema de arquivos. Os modos de abertura podem ser o 'w' (write) ou 'a' (append).

```
with open('arquivo.txt', 'w') as arquivo:  
    arquivo.write("Olá, mundo!\n")  
    linhas = ['Linha 1', 'Linha 2']  
    arquivo.writelines(linhas)
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Modos de escrita:

- **write()**: escreve uma string no arquivo.
- **writelines()**: escreve uma lista de strings no arquivo.

É importante lembrar que:

'w' (write): abre o arquivo para escrita. Se o arquivo já existir, ele será truncado (conteúdo anterior será removido) antes de escrever. Se o arquivo não existir, um novo será criado.

'a' (append): abre o arquivo para escrita, mas adiciona novos conteúdos ao final do arquivo, sem truncá-lo.

Exposição

Fechamento automático do arquivo

O bloco **with** é utilizado para garantir que o arquivo seja fechado corretamente após a manipulação. Isso é importante para liberar recursos do sistema. Observe os dois exemplos abaixo:

Exemplo 1:

```
# Escrita em um arquivo com fechamento automático usando 'with'
conteudo_para_escrita = "Conteúdo a ser escrito no arquivo."

with open('arquivo_com_fechamento.txt', 'w') as arquivo:
    arquivo.write(conteudo_para_escrita)
# O arquivo é fechado automaticamente ao sair do bloco with
```

Exemplo 2:

```
# Escrita em um arquivo sem fechamento automático
conteudo_para_escrita = "Conteúdo a ser escrito no arquivo."

arquivo = open('arquivo_sem_fechamento.txt', 'w')
arquivo.write(conteudo_para_escrita)
# É necessário fechar explicitamente o arquivo
arquivo.close()
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

- No segundo exemplo, usa-se **open()** sem **with**, exigindo fechar o arquivo com **close()** manualmente. Isso libera recursos.
- Porém, o uso de **with**, como no primeiro exemplo, é mais seguro e organizado, evitando esquecimentos.

Manipulação de arquivos binários

Se você estiver lidando com arquivos binários, adicione um **'b'** ao modo. Por exemplo, **'rb'** para leitura binária.

```
with open('imagem.jpg', 'rb') as arquivo_binario:  
    dados = arquivo_binario.read()  
    # Processar os dados binários
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



Recomendação

Lembre-se de que é uma boa prática **fechar o arquivo após a manipulação** ou usar o bloco **with** para fechamento automático. A manipulação de arquivos é crucial para a persistência de dados em programas Python.

Tratamento de exceções

O tratamento de exceções é a prática de **gerenciar erros ou situações inesperadas** durante a execução de programas de maneira controlada.

Viabiliza que o código responda a esses eventos de maneira apropriada, evitando que o programa seja encerrado abruptamente.

Usam-se blocos **try**, **except**, **else** e **finally** para implementá-lo.

Como funciona:

No bloco **try**, insere-se o código que pode gerar uma exceção. Se uma exceção surgir, o controle passa para o **except**, que lida com o erro.

O bloco **else**, opcional, executa o código se o **try** não falhar.

O bloco **finally**, também opcional, roda código independentemente de ocorrer uma exceção ou não.

Tratamento de exceções

De modo geral, sua estrutura é:

```
try:
    # Código que pode levantar uma exceção
    # ...
except TipoDeExcecao as nome_da_excecao:
    # Código a ser executado se uma exceção do tipo TipoDeExcecao for Levantada
    # ...
except OutroTipoDeExcecao as outro_nome:
    # Código para outro tipo de exceção
    # ...
else:
    # Código a ser executado se nenhum erro ocorrer no bloco try
    # ...
finally:
    # Código que será executado sempre, independentemente de ocorrer uma exceção ou não
    # ...
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

- Bloco **try** verifica erros. Sem erros, o **except** é ignorado.
- Cada **except** lida com uma exceção específica, executando seu código se ocorrer.
- O **else**, opcional, roda se o **try** for bem-sucedido.
- O **finally**, também opcional, executa sempre, sendo ideal para limpeza ou liberação de recursos.

Tratamento de exceções

Exemplo:

```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
    print("Resultado:", resultado)
except ValueError:
    print("Você não digitou um número válido.")
except ZeroDivisionError:
    print("Não é possível dividir por zero.")
else:
    print("Nenhuma exceção ocorreu.")
finally:
    print("Este bloco sempre será executado.")
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

No exemplo acima, o código tenta converter a entrada do usuário em um número inteiro, realiza uma divisão e lida com exceções específicas, como **ValueError** (quando a entrada não é um número) ou **ZeroDivisionError** (quando a divisão por zero ocorre).



Tome nota

O **tratamento de exceções** é uma prática essencial para escrever um código **Python confiável**, como em situações em que eventos imprevisíveis podem ocorrer durante a execução do programa.

Tratamento de exceções

Benefícios do tratamento de exceções



Elaborado especialmente para o curso com imagem © Getty Images.



Vamos
fazer um
quiz

**Qual é a função principal da
função `open()` ao lidar com
arquivos em Python?**

Ler um arquivo existente.

**Fechar um arquivo
automaticamente.**

**Abrir um arquivo para
escrita ou leitura.**

**Modificar o conteúdo de
um arquivo.**



Vamos
fazer um
quiz

**Qual é a função principal da função
`open()` ao lidar com arquivos em
Python?**



Ler um arquivo existente.

**Fechar um arquivo
automaticamente.**



**Abrir um arquivo para
escrita ou leitura.**

**Modificar o conteúdo de
um arquivo.**



FEEDBACK GERAL DA ATIVIDADE

A função `open()` é utilizada para abrir um arquivo no Python, e ela suporta diferentes modos, como leitura ('r'), escrita ('w'), e anexação ('a')



Vamos
fazer um
quiz

Qual é a diferença principal entre os modos de abertura 'w' e 'a' ao se usar a função `open()` para escrever em um arquivo?

O modo 'w' adiciona conteúdo ao final do arquivo, enquanto o modo 'a' sobrescreve o arquivo.

O modo 'a' adiciona conteúdo ao final do arquivo, enquanto o modo 'w' sobrescreve o arquivo.

Ambos os modos adicionam conteúdo ao final do arquivo.

Ambos os modos sobrescrevem o arquivo.



Vamos
fazer um
quiz

Qual é a diferença principal entre os modos de abertura 'w' e 'a' ao se usar a função `open()` para escrever em um arquivo?



O modo 'w' adiciona conteúdo ao final do arquivo, enquanto o modo 'a' sobrescreve o arquivo.

O modo 'a' adiciona conteúdo ao final do arquivo, enquanto o modo 'w' sobrescreve o arquivo.



Ambos os modos adicionam conteúdo ao final do arquivo.

Ambos os modos sobrescrevem o arquivo.



FEEDBACK GERAL DA ATIVIDADE

O modo 'w' (write) sobrescreve o arquivo existente, enquanto o modo 'a' (append) adiciona conteúdo ao final do arquivo existente.



Vamos
fazer um
quiz

Por que é recomendável usar o bloco with ao lidar com arquivos em Python?

Porque o bloco with garante a leitura de arquivos em binário.

Porque o bloco with é obrigatório para a escrita de arquivos em Python.

Porque o bloco with facilita a formatação de strings para a escrita em arquivos.

Porque o bloco with simplifica o tratamento de exceções e garante o fechamento automático do arquivo.



Vamos
fazer um
quiz

Por que é recomendável usar o bloco with ao lidar com arquivos em Python?



Porque o bloco with garante a leitura de arquivos em binário.

Porque o bloco with é obrigatório para a escrita de arquivos em Python.



Porque o bloco with facilita a formatação de strings para a escrita em arquivos.

Porque o bloco with simplifica o tratamento de exceções e garante o fechamento automático do arquivo.



FEEDBACK GERAL DA ATIVIDADE

O bloco with é utilizado para garantir que os recursos, como arquivos, sejam fechados automaticamente ao final do bloco, evitando vazamentos de recursos e simplificando o tratamento de exceções relacionadas com a manipulação de arquivos.



© Getty Images

O que nós
**aprendemos
hoje?**

Hoje desenvolvemos:

- 1** A compreensão sobre o conceito de arquivos e os diversos formatos e características que eles podem ter.
- 2** O entendimento dos conceitos de abertura, leitura e escrita dos arquivos com Python.
- 3** A percepção dos benefícios do tratamento de exceções, que é a prática de gerenciar erros ou situações inesperadas durante a execução de programas de maneira controlada.



Saiba mais

Trabalhar com arquivos e diretórios no Python é algo essencial! Descubra como:

FELIPE, A. Trabalhando com arquivos e diretórios no Python. *Alura*, 14 jun. 2017. Disponível em: <https://www.alura.com.br/artigos/trabalhando-com-arquivos-e-diretorios-no-python>. Acesso em: 5 abr. 2024.

E você já ouviu falar sobre JSON? Entenda como utilizar esse formato no curso do Alura.

LEITE, G. Saiba o que é JSON e como utilizar. *Alura*, 5 set. 2023. Disponível em: <https://www.alura.com.br/artigos/o-que-e-json>. Acesso em: 5 abr. 2024.

Referências da aula

FELIPE, A. Trabalhando com arquivos e diretórios no Python. *Alura*, 14 jun. 2017. Disponível em: <https://www.alura.com.br/artigos/trabalhando-com-arquivos-e-diretorios-no-python>. Acesso em: 5 abr. 2024.

LEITE, G. Saiba o que é JSON e como utilizar. *Alura*, 5 set. 2023. Disponível em: <https://www.alura.com.br/artigos/o-que-e-json>. Acesso em: 5 abr. 2024.

MENEZES, N. N. C. *Introdução à programação com Python: algoritmos e lógica de programação para iniciantes*. São Paulo: Novatec, 2019.

PYTHON. 3.12.2 *Documentation*. O tutorial de Python. 8. Erros e exceções. Disponível em: <https://docs.python.org/pt-br/3/tutorial/errors.html>. Acesso em: 5 abr. 2024.

PYTHON. 3.12.2 *Documentation*. The Python Standard Library. Built-in Functions. Disponível em: <https://docs.python.org/3/library/functions.html>. Acesso em: 5 abr. 2024.

Identidade visual: Imagens © Getty Images

Educação
Profissional
Paulista

Técnico em
Ciência de
Dados